

Resumo de estudo: IA e Aprendizagem de Máquina II

Resumo das aulas 00 a 17 da disciplina (Prof. João Paulo Orlando), organizado pelos temas da ementa. Cada seção termina com o que mais provavelmente vira pergunta. A última parte cobre os itens da ementa que as aulas trataram pouco ou nada.

1. Fundamentos (Aula 01)

Aprendizado de máquina é treinar um algoritmo com dados para que ele faça previsões usadas em decisões. Faz sentido quando existe mapeamento claro entre entrada e saída, muitos dados disponíveis, tolerância a erro e o problema não muda rápido demais.

Etapas de um projeto: definir objetivo, coletar dados, explorar e preparar os dados (a etapa que mais consome tempo), escolher o tipo de aprendizado, treinar com divisão treino/teste, avaliar, ajustar hiperparâmetros e retreinar.

Tipos de aprendizado vistos em aula:

- **Supervisionado:** dados rotulados. Se o alvo é discreto, é classificação (vai chover ou não); se é contínuo, é regressão (quantos milímetros).
- **Não supervisionado:** dados sem rótulo, o algoritmo procura estrutura por conta própria (agrupamento, associação, redução de dimensionalidade).
- **Por reforço:** o agente aprende por recompensa, e a decisão depende do estado atual.

Erros clássicos:

- **Bias (viés):** modelo treinado com dados desiguais ou incompletos fica tendencioso.
- **Underfitting:** o modelo não aprende nem o treino. **Overfitting:** decora o treino e erra em dados novos.
- **Leakage (vazamento):** usar no treino informação que não existiria no momento da previsão. É sutil e difícil de detectar.

Provável pergunta: diferença entre classificação e regressão; o que é overfitting e como perceber (erro de treino baixo e de teste alto); o que é vazamento de dados.

2. Ferramentas (Aulas 02 a 04)

NumPy (arrays e operações vetoriais), Pandas (DataFrame, leitura de CSV, filtros, `describe`, `dropna`) e Matplotlib/Seaborn (dispersão, histograma, heatmap de correlação). Não costumam cair como conceito, mas é bom saber que o `heatmap de df.corr()` é a forma rápida de achar features redundantes.

3. Análise exploratória e seleção de características (Aula 05)

EDA expõe as características principais dos dados com estatística e gráficos, e é o primeiro passo da engenharia de features. O **Quarteto de Anscombe** justifica a etapa: quatro conjuntos com médias, variâncias e correlação praticamente iguais e formatos completamente diferentes. Estatística resumida sem gráfico engana.

Correlação não é causalidade.

- **Pearson:** mede relação **linear**, vai de -1 a 1, não depende de unidade.
- **Spearman:** não paramétrica, mede relação **monotônica** usando postos, serve para dados ordinais.

Como ler a correlação:

- Entrada com saída alta: feature útil. Se for alta demais, desconfie de vazamento.
- Entrada com saída baixa: feature pouco útil, risco de underfitting.
- Entrada com entrada alta: **multicolinearidade**. Redundância que deixa os coeficientes da regressão linear instáveis e difíceis de interpretar. Mantenha só uma das duas.

Métodos de seleção de características:

- **All-in:** usa todas, quando a EDA já foi bem feita.
- **Backward elimination:** começa com todas e remove, uma por vez, a de maior p-valor acima do limite (geralmente 5%).
- **Forward selection:** começa com nenhuma e adiciona, uma por vez, a que mais melhora o modelo.
- **Bidirectional:** alterna forward e backward.
- **Score comparison:** testa todas as $2^N - 1$ combinações e fica com a melhor. Exato, mas caro.

Existe ainda a importância de features que árvores e Random Forest calculam sozinhas (`feature_importances_`), vista nas aulas 9 e 10.

Provável pergunta: por que não confiar só na estatística descritiva; diferença entre Pearson e Spearman; o que é multicolinearidade e por que atrapalha.

4. Pré-processamento (Aula 06)

- **Valores faltantes:** preencher com a **média** (numéricos) ou a **moda** (categóricos) via `SimpleImputer`. Em datasets grandes, também dá para remover as linhas, se fizer sentido.
- **Variáveis categóricas:** **LabelEncoder** quando há ordem natural (faixas de renda A a E); **OneHotEncoder** quando não há ordem (estado de nascimento), criando uma coluna binária por categoria. Usar **LabelEncoder** em categoria sem ordem inventa

uma relação numérica que não existe.

- **Divisão treino/teste** com `train_test_split`, tipicamente 80/20 ou 2/3 e 1/3.
- **Escalação** (`StandardScaler`): sempre **depois** da divisão. O `fit` é feito só no treino e o teste recebe apenas `transform`. Ajustar o scaler com dados de teste é vazamento.

Provável pergunta: por que o scaling vem depois do split; quando usar one-hot e quando usar label encoding.

5. Regressão (Aulas 07 a 09)

Regressão linear simples: $y = b_0 + b_1 \cdot x + e$. Pressupõe relação linear, variância homogênea do erro e observações independentes. **Múltipla:** várias features, ajusta um hiperplano, e aqui a multicolinearidade pesa. **Polinomial:** cria x^2 , x^3 ... como novas features e continua sendo um modelo linear nos coeficientes. Grau alto demais leva a overfitting.

Métricas de regressão:

Métrica	O que mede	Leitura
R^2	proporção da variância explicada	quanto mais perto de 1, melhor
MAE	média do erro absoluto	na unidade do alvo, robusto a outliers
RMSE	raiz da média do erro ao quadrado	na unidade do alvo, pune mais os erros grandes

Árvore de decisão para regressão: divide o espaço das features em regiões escolhendo os cortes que mais **reduzem a variância**; a previsão é a média da folha. É gulosa (cada divisão depende da anterior), por isso não garante a árvore ótima global. **Não precisa de escalonamento**, porque cada corte compara uma feature com um limiar. Sem limite de profundidade ela decora os dados; `max_depth` é o hiperparâmetro que controla isso. Vantagem forte: interpretabilidade.

Random Forest: ensemble de árvores por **bagging**. Cada árvore treina com uma amostra diferente dos dados e um subconjunto das features; a previsão final é a **média** (regressão) ou a **votação** (classificação).

Validação cruzada (k-fold): divide os dados em k partes, treina k vezes usando uma parte diferente como teste em cada rodada e tira a média. Estimativa mais confiável que um único split.

Parâmetro x hiperparâmetro: parâmetros o modelo aprende no treino (coeficientes); hiperparâmetros quem define é o desenvolvedor (`max_depth`, `n_estimators`, K do KNN). **GridSearchCV** testa combinações de hiperparâmetros com validação cruzada e devolve a melhor.

Provável pergunta: $R^2 \times MAE \times RMSE$; por que a árvore dispensa scaling; como a Random Forest reduz o overfitting da árvore única; o que é validação cruzada.

6. Classificação (Aulas 10 a 12)

Árvore e Random Forest para classificação: mesma lógica da regressão, mas os cortes buscam reduzir a **entropia** (desordem das classes) e maximizar o **ganho de informação**.

SVM (Support Vector Machine): procura o hiperplano que separa as classes com a **maior margem** possível. Os pontos mais próximos da fronteira são os **vetores de suporte**, e só eles definem o modelo.

- **Hard margin:** dados perfeitamente separáveis.
- **Soft margin:** aceita alguns erros (variáveis de folga), útil quando há sobreposição.
- **Kernel trick:** leva os dados para uma dimensão maior, onde ficam separáveis, sem calcular essa transformação explicitamente. Kernels: linear, polinomial, RBF (gaussiano, uso geral) e sigmoide. A escolha do kernel e dos parâmetros é decisiva, então use GridSearch.
- **Multiclasse:** one-vs-rest (cada classe contra todas as outras) ou one-vs-one (um classificador para cada par, vence a mais votada).
- **Exige escalonamento**, porque a margem depende de distância.

KNN (K-Nearest Neighbors): classifica pelo voto dos K vizinhos mais próximos. Passos: escolher K (ímpar, para evitar empate), medir a distância (Minkowski com $p=2$ é a euclidiana, $p=1$ é a Manhattan), contar as classes dos vizinhos e atribuir a mais votada. É um **aprendizado preguiçoso e baseado em instâncias**: não constrói modelo, guarda os dados e faz todo o trabalho na previsão. **Exige escalonamento**.

Provável pergunta: o que são vetores de suporte; para que serve o kernel; por que SVM e KNN precisam de scaling e árvore não; por que o KNN é chamado de preguiçoso.

7. Métricas de classificação (Aulas 10 e 13)

A **matriz de confusão** cruza real x previsto: VP, VN, FP, FN.

Métrica	Fórmula	Responde
Acurácia	$(VP+VN)/total$	quanto acertei no geral
Precisão	$VP/(VP+FP)$	dos que previ positivos, quantos eram
Recall (sensibilidade)	$VP/(VP+FN)$	dos positivos reais, quantos encontrei
Especificidade	$VN/(VN+FP)$	dos negativos reais, quantos encontrei
F1	média harmônica de precisão e recall	equilíbrio entre os dois

Paradoxo da acurácia: com 9.650 negativos e 150 positivos, um modelo que responde sempre "negativo" tem 98,5% de acurácia e é inútil. Em dados desbalanceados é preciso olhar matriz de confusão, recall e F1.

Quando priorizar cada uma: precisão quando o **falso positivo** custa mais (recomendar investimento ruim); recall quando o **falso negativo** custa mais (liberar para viagem uma pessoa infectada, deixar passar uma doença).

Provável pergunta: paradoxo da acurácia; dar um exemplo em que recall importa mais que precisão; o que o F1 resume.

8. Aprendizado não supervisionado e clusterização (Aulas 14 e 15)

Três tarefas principais: **clusterização** (agrupar por semelhança), **associação** (regras do tipo "quem compra pão compra leite", análise de cesta) e **redução de dimensionalidade**. O algoritmo cria as próprias categorias, e elas podem não ser as que você esperava.

K-means:

1. Escolher K.
2. Posicionar K centroides iniciais.
3. Atribuir cada ponto ao centroide mais próximo.
4. Recalcular cada centroide como a média dos pontos do seu grupo.
5. Repetir 3 e 4 até os centroides pararem de mudar.

Escolha de K pelo método do cotovelo: calcula o **WCSS** (soma das distâncias ao quadrado de cada ponto ao seu centroide) para vários K. O WCSS sempre cai com mais clusters e chega a zero quando K é igual ao número de pontos, por isso o critério é o "cotovelo", onde a queda deixa de valer a pena.

Armadilha da inicialização aleatória: centroides iniciais ruins levam a um agrupamento ruim (mínimo local). O **k-means++** resolve escolhendo cada novo centroide com probabilidade proporcional à distância do centroide mais próximo, espalhando os pontos iniciais.

Cuidado para não confundir: **KNN** é supervisionado e classifica; **K-means** é não supervisionado e agrupa. Os dois usam distância.

Provável pergunta: passo a passo do K-means; como escolher K; diferença entre KNN e K-means.

9. Redução de dimensionalidade: PCA (Aula 16)

Seleção de features escolhe algumas das colunas originais; **extração** cria colunas novas combinando as originais. O PCA é extração e é não supervisionado.

Passo a passo: escalonar e centralizar os dados; calcular autovetores e autovalores da matriz de covariância (ou usar SVD); ordenar os autovalores do maior para o menor e escolher os k primeiros; projetar os dados nessas direções. Os componentes resultantes são **não correlacionados** entre si.

A **variância explicada** (`explained_variance_ratio_`) diz quanta informação cada componente guarda, e é o critério para escolher quantos manter. Reduz ruído, custo e multicolinearidade, e permite visualizar dados de muitas dimensões, mas os componentes perdem interpretabilidade e nem sempre melhoram o modelo.

Provável pergunta: diferença entre seleção e extração de características; por que escalonar antes do PCA; o que é variância explicada.

10. Sistemas de múltiplos classificadores: ensembles (Aulas 09 e 17)

	Bagging	Boosting
Treino	modelos em paralelo, cada um numa amostra diferente	modelos em sequência, cada um corrigindo o anterior
Combinação	votação ou média	soma ponderada
Reduz principalmente	variância (overfitting)	viés (underfitting)
Exemplo	Random Forest	AdaBoost, XGBoost

Gradient Boosting / XGBoost: começa com um modelo simples; calcula os resíduos (real menos previsto); treina uma nova árvore para prever esses resíduos; soma ao modelo e repete. O XGBoost é uma implementação otimizada e regularizada disso, com histórico forte em competições de dados tabulares.

Provável pergunta: diferença entre bagging e boosting; por que combinar vários modelos fracos funciona.

11. Temas da ementa que as aulas trataram pouco

Esses itens estão na ementa, mas não têm aula própria. Como o avaliador é seu orientador, os exemplos do TCC são a forma mais natural de responder.

- **Semi-supervisionado:** poucos dados rotulados e muitos sem rótulo; o modelo aprende com os rotulados e usa isso para rotular o resto.
- **Online x em lote:** em lote, o modelo treina uma vez com todo o conjunto e depois é só usado (o TCC é assim). Online, aprende continuamente com dados chegando em fluxo.
- **Baseado em instâncias x baseado em modelo:** instâncias guardam os exemplos e comparam na hora (KNN); modelo generaliza em parâmetros e descarta os dados (regressão, árvore, SVM, a rede do TCC).
- **Treino, validação e teste:** o treino ajusta parâmetros, a validação escolhe hiperparâmetros e detecta overfitting, o teste só é usado no fim. No TCC: 80/10/10.
- **Amostragem estratificada:** mantém a proporção das classes em cada partição, essencial com classes desbalanceadas. O TCC usa divisão estratificada.
- **Evitar overfitting:** validação cruzada, regularização (penaliza coeficientes grandes), **early stopping** (para quando o erro de validação começa a subir), menos complexidade (`max_depth`) e mais dados. No TCC, a loss de validação subindo no piloto com 4 mil exemplos foi o sinal de overfitting, resolvido com o dataset completo.
- **Problemas com os dados:** insuficientes, não representativos, de baixa qualidade (ruído, faltantes, outliers) e features irrelevantes. O TCC tem exemplos dos dois primeiros.
- **Bias:** dado desbalanceado gera modelo tendencioso para a classe majoritária. É o motivo de usar métricas macro no TCC.
- **Análise estatística de resultados:** comparar modelos com a mesma divisão e métricas adequadas, olhar média e desvio da validação cruzada, comparar com baseline. No TCC, a comparação zero-shot, few-shot e fine-tuned cumpre esse papel.
- **Deep learning:** redes neurais com muitas camadas que aprendem as próprias features a partir de dados brutos. Precisam de muitos dados e GPU. O TCC é o seu exemplo: modelo transformer ajustado por QLoRA.
- **Reconhecimento de padrões:** nome geral para classificar ou agrupar dados a partir de regularidades. Todas as técnicas acima são formas de fazer isso.